UNIVERSIDAD NACIONAL DE LUJÁN

Departamento de Ciencias Básicas • Lic. en Sistemas

Programación Orientada a Objetos

(11[0|2]76)

Clase 06 • **Ejercicio 01** • Ciclo Lectivo 2026

MATERIAL PARA ESTUDIANTES
ENUNCIADO PRÁCTICO



Ejercicio 1 (Live Coding): Tienda de Ropa y Precios Polimórficos

Contexto

En este ejercicio nos introduciremos en la **Herencia y el Polimorfismo**, reemplazando condicionales múltiples (*code smells*) por sobreescritura de métodos (`@Override`) que respeten el Principio de Sustitución de Liskov (LSP).

Enunciado

Una tienda de ropa gestiona la venta de sus prendas.

Todas las **Prendas** (`Prenda`) poseen un nombre y un precio base. Sin embargo, existen subclases especializadas de prendas con reglas de precio específicas:

- Una **Remera** (`Remera`) suma un recargo fijo por estampado si es declarada como importada.
- Un **Pantalón** (`Pantalon`) aplica un descuento del 10% sobre su precio base si pertenece a una temporada anterior.

Una **Venta** (`Venta`) debe almacenar prendas genéricas en una lista y calcular el total cobrado sumando el precio final de cada prenda mediante polimorfismo.

Tareas:

1. Diseñe la superclase `Prenda` declarando el método `calcularPrecioFinal()`.
2. Diseñe las subclases `Remera` y `Pantalon` utilizando `@Override` para especializar el cálculo del precio final, reutilizando la lógica base cuando sea posible.
3. Diseñe la clase `Venta` conteniendo una colección de `Prenda` e implemente el método `calcularTotal()` sin utilizar comprobaciones de tipo (`instanceof` o `switch`).

Restricciones y Tips

- **Prohibición de Condicionales de Tipo:** Está estrictamente prohibido utilizar `instanceof`, `getClass()` o `switch` dentro de `Venta` para preguntar qué tipo de prenda es. Confíe en el despacho dinámico de métodos (polimorfismo).
- **Uso de `@Override`:** Utilice siempre la anotación `@Override` en las subclases para garantizar que la firma coincide exactamente con el método de la superclase.

Preguntas Conceptuales

Al finalizar el ejercicio, reflexione sobre las siguientes preguntas:

1. ¿Por qué es un mal diseño utilizar un condicional `if (prenda instanceof Remera)` para calcular el total de la venta en lugar de invocar `prenda.calcularPrecioFinal()`?
2. ¿De qué manera el Principio de Sustitución de Liskov (LSP) nos permite agregar nuevos tipos de prendas en el futuro sin modificar ni una sola línea de la clase `Venta`?